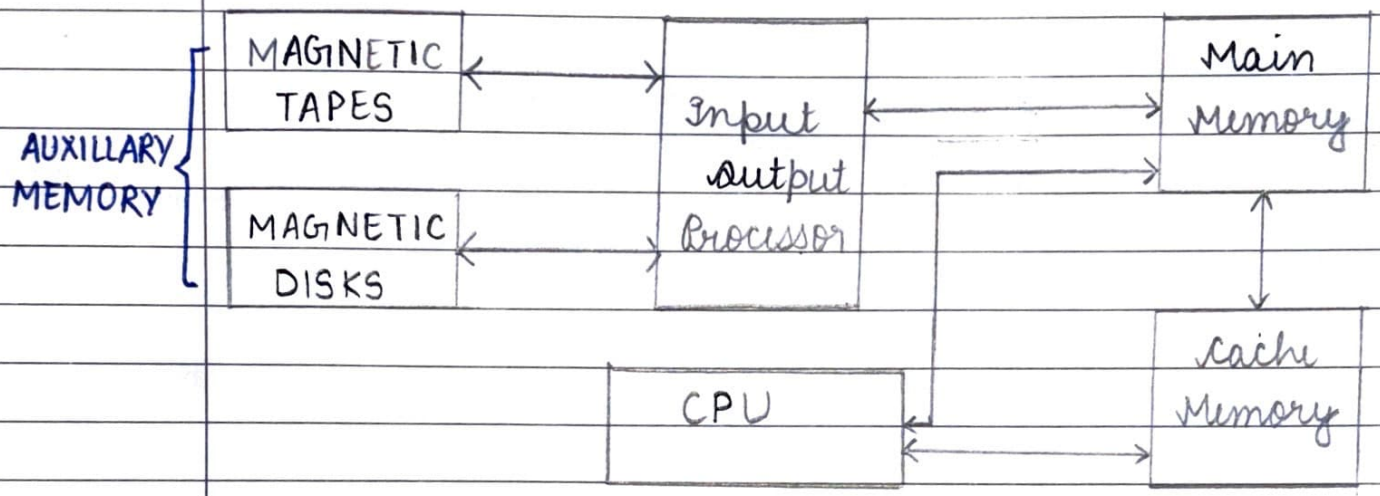
Topic _____ Date _____

→ **Memory Hierarchy**

It consists of all storage devices employed in a from slow but high capacity auxillary memory to faster main memory or even smaller cache memory



- At the bottom of hierarchy are slow magnetic tapes used to store removable files
- Magnetic disks are used as backup storage
- Main memory occupies central position by being able to communicate directly with CPU and auxillary memory devices through I/O processor
- When programs absent in main memory are needed by CPU, they are brought from the auxillary memory via I/O processor
- When programs present in main memory aren't needed by CPU, they're sent to the aux. memory to make space to store current programs & data
- Cache is used to increase the speed of processing by making data available to CPU at a rapid rate.

Citizen Prime

Topic _____ Date _____

- Cache is used to store segments of programs currently being executed in CPU & temporary data frequently needed in present calculation
- CPU has direct access to both cache and main memory but not auxillary memory

	Type of memory	Speed	Cost	Storage
1.	Auxillary	Low	Low	High
2.	Cache	High	High	Small

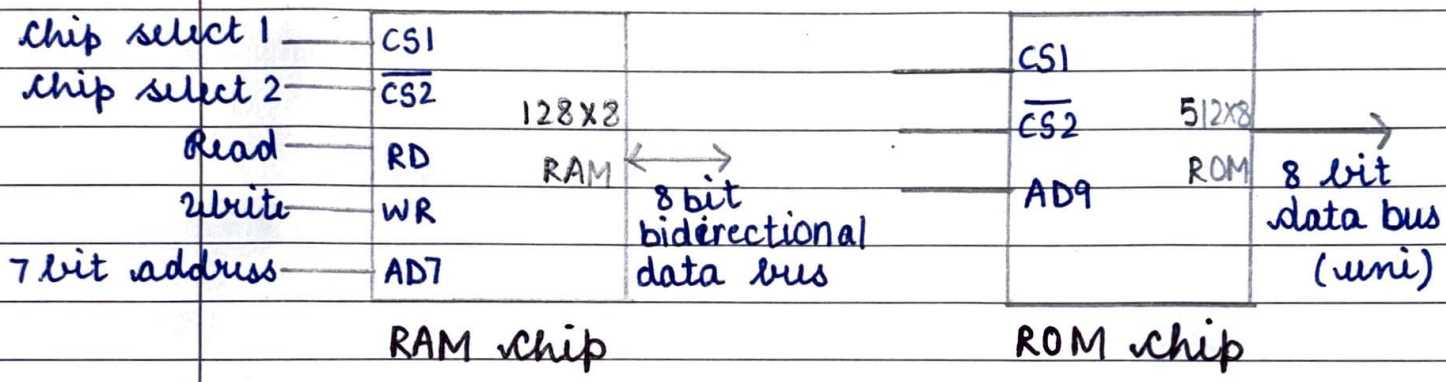
1. Main Memory
   - The main memory is the central storage unit in the computer system. It's relatively large, fast memory used to store programs & data
   - Principle of main memory is based on the integrated semiconductor circuits
   - RAM
     - > It's available in two possible operating modes static & dynamic
     - > SRAM consists essentially of internal flip flop that store the binary information, the stored information remains valid as long as power is applied to the unit.
     - > DRAM stores binary info in form of electrical charges that are applied to capacitors. The stored charge on capacitors tends to discharge with time & capacitors must be periodically recharged by refreshing the memory.
     - > DRAM offers reduced power consumption & large storage capacity in a single chip while SRAM is easier to use & has shorter read & write cycles.

Citizen Prime

Topic _____ Date _____

- **ROM**
  - > It's used for storing permanent programs that don't change once the production of computer is completed
  - > It stores bootstrap loader which is a program used to start computer software operating when power is turned on.

→ **RAM and ROM chips.**



1. **RAM chips**
   - They use bidirectional databus allowing data transfer in both direction (Memory ↔ CPU) during read & write operations
   - They use three state buffers hence the out put can be high / low / high impedance
   - uses chip select signals CS1 and $\overline{CS2}$ to enable the chip only when needed.

2. **ROM chips**
   - can only read data, so it's databus operates strictly as an output
   - can store more bits in a smaller physical space compared to RAM

Citizen Prime

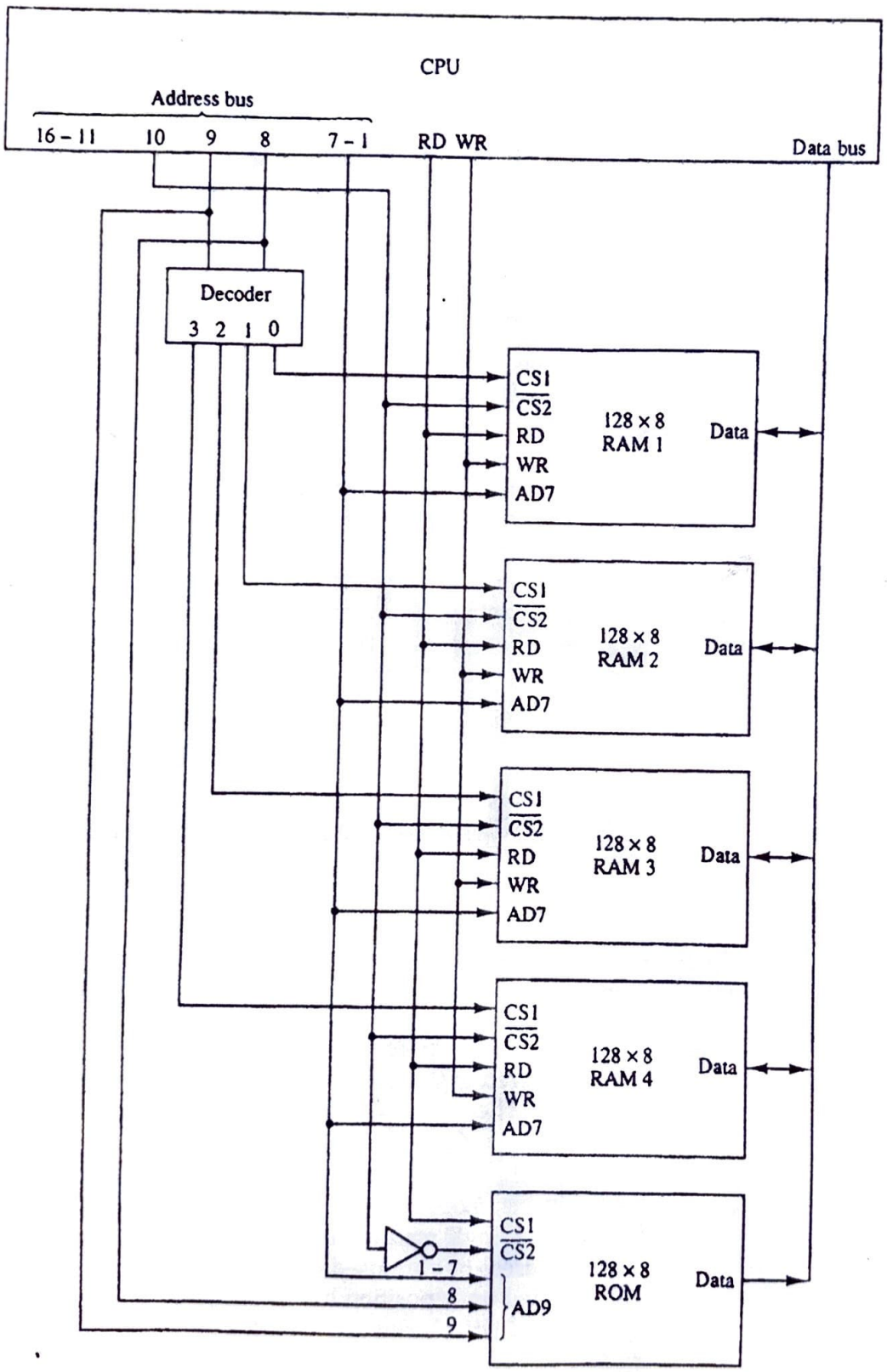


Figure 12-4 Memory connection to the CPU.

Topic _____ Date _____

→ Working of a RAM chip

CS1	$\overline{CS2}$	RD	WR	Function	State of Bus
0	0	X	X	Inhibit	High imp.
0	1	X	X	Inhibit	High imp.
1	0	0	0	Inhibit	High imp.
1	0	0	1	Write	Input to RAM
1	0	1	X	Read	Output from RAM
1	1	X	X	Inhibit	High imp.

- A RAM chip is active only when CS1 = 1 & $\overline{CS2}$ = 0
- If chip signals are inactive / if neither read or write is enabled, the chip is inhibited and databus remain in high impedence state (acts like an open circuit.
- When WR is enabled, data from data bus is stored at location specified by address lines.
- When RD is enabled, the byte from slected memory location is placed onto data bus

→ Memory Address Map

- It's a table / diagram that shows the exact allocation of hexadecimal code address assigned to each RAM & ROM chip.
- It uses lower order and higher order address lines to differentiate between data & chips.

→ CPU to memory connection

- A 2X4 decoder uses address lines 8 & 9 to generate slect signals for 4 individual

Citizen Prime

Topic _____ Date _____

RAM chips

- Address lines 10 acts as a master switch when line 10 is 0, a RAM chip is selected when line 10 is 1, a ROM chip is selected

→ Auxiliary memory

- Magnetic disks and magnetic tapes are most common auxiliary (secondary) storage devices used in computer systems
- Magnetic drums, magnetic bubble memory & optical disks are also used but less frequently.

→ Performance parameters of auxiliary memory

1. Access time : The average time required to reach a specific storage locations in the memory & retrieve it's contents.
2. Seek time : The time needed to physically position the read - write head over the target track / location.
3. Transfer time : The time required to transfer data to or from the device
4. Transfer rate : The number of characters or words, the device can transfer per second once the positioning is complete

→ Magnetic drum v/s magnetic disks

- Drum is a rotating cylinder whereas disk is flat, round circular plate
- Both use high speed rotating surfaces coated with magnetic material & records bits as magnetic spots are detected by read/write heads

Citizen Prime

Topic _____ Date _____

- A magnetic disk provides a larger recording surface area than a magnetic drum of equal physical size allowing it to store significantly more data.

→ Magnetic Disks

- Made up of circular metal / plastic plates coated with magnetic material
- Multiple disks can be stacked on a single rotating spindle with read / write heads for each surface
- It uses tracks (concentric circles where bits are stored) & sectors (sub-division of tracks & is the minimum unit of information transferred in single operation.
- A single read / write head per surface moves mechanically across tracks using address guided assembly.
- There are separate read / write heads for each track. Tracks are selected electronically making it faster but more expensive
- Hard disks are permanently attached whereas floppy disks are removable

→ Magnetic Tapes

- Consists of flexible plastic strips coated with magnetic recording medium and is managed by electric & magnetic controls
- Bits are recorded as magnetic spots across parallel tracks. Typically 7/9 bits are written simultaneously to form characte

Citizen Prime

Topic _____ Date _____

- Since tape drives can't start/stop instantly between individual characters, data is written is blocks called records
- Unrecorded spaces called gaps are inserted between records so the tape can accelerate to constant operating speed / safely come to a stop.

→ Associative Memory

- In traditional memory, data is found by looking up it's specific physical address using search algorithms, which can take multiple memory accesses
- Associative memory is accessed using the data's content itself rather than an address.
- It searches all memory locations simultaneously and in parallel, which makes searching extremely fast.
- Because every single storage cell requires extra hardware logic to match data contents associative memory is more expensive than standard RAM

→ Hardware Organization

- An associative memory system is built using specific registers & arrays :-

1. Memory Array (M×N) : consists of m words with 'n' bits per word.
2. Argument Register (A) : Holds the 'n' bit data pattern or word that you want to search for.
3. Key Register (K) : Acts as a mask to select

Citizen Prime

Topic _____ Date _____

specific fields / particular bits within the argument word to focus the search.

4. Match Register (M): contains 'm' bits (one for each word in memory); if a stored word matches the argument, its corresp. bit in match register is set to 1.

→ Cell structure & match logic

1. Cell components: Each memory cell (C_{ij}) has a flip flop storage element (F_{ij}), reading or writing circuits and a matching logic

2. Bit equality (x_j): The match logic compares an argument bit (A_j) with a stored bit (F_{ij}). Their equality is expressed as

$$x_j = A_j F_{ij} + \overline{A_j}\,\overline{F_{ij}}$$

$x_j = 1$, if they match and $x_j = 0$ if they don't match.

3. Word Match (M_i): For an entire word i to match, all its individual bits must match which is the logical AND operation of all x_j variables.

$$M_i = x_1 x_2 x_3 \cdots x_n$$

→ Key Register & masking logic

1. Role of key bits (K_j): The key register allows you to ignore certain bits during comparison
   - If $K_j = 0$, that bit position is masked out and ignored
   - If $K_j = 1$, bit must be compared

2. Combined match formula: The complete match logic incorporating the key bit is represented as

$$M_i = \prod_{j=1}^{n} (A_j F_{ij} + \overline{A_j}\,\overline{F_{ij}} + \overline{K_j})$$

Citizen Prime

TOPIC _____ DATE _____

→ Read & Write Operations

1. Read operation : When multiple words match the search criteria, their match bits are set to 1. The system then scans the match register to read the matched words sequentially connecting output Mi directly to read lines can also automatically output the data without needing special read commands.

2. Write operation: Data can be loaded into associative memory either all at once or sequentially. Using address decoders, writing can be performed like RAM where $m = 2^d$ address lines are used.

→ Tag Register

- A special register called tag register is used to keep track of which memory words are currently active & which are empty
- While deleting old words or inserting new ones, the system scans the tag register to instantly find available empty spaces (where tag bit is 0)

Cache Memory

→ Locality of Reference

Programs tend to reference memory locations in localized areas over the short intervals (due to loops, subroutine and table lookups) rather than scattering references randomly.

TOPIC _____ DATE _____

→ Purpose of cache memory

Because of locality, placing active program instructions and data into a small, extremely fast memory placed between the CPU and main memory significantly reduces average access time & total execution time.

→ Hit ratio

- When the CPU finds a word in the cache, it is a hit, if not it is a miss & must be fetched from main memory
- The hit ratio is the number of hits divided by the total references to memory often reaching 0.9 or higher in typical program

→ Cache mapping procedures

- When a word is requested from the main memory, a mapping process determines how it is stored in cache
- The three main techniques are :-

1. Associative Mapping
2. Direct Mapping
3. Set Associative mapping

→ Associative Mapping

- It's the fastest and most flexible cache org. An associative memory stores both the address (tag) and the actual data word.
- The CPU places a 15 bit address into an argument register & the entire cache is searched in parallel to find a matching address.

TOPIC _____ DATE _____

If the cache is full when a miss occurs, a replacment policy (such as FIFO) is used to clear an un-needed pair & make room for the new one.

→ Direct Mapping

- Instead of expensive associative memory, the direct mapping uses a standard RAM for cache
- A 15 bit main memory address is split into two parts i.e. a tag field & an index field. The index field is used to access cache directly.
- The tag of the CPU address is compared with the tag stored in the cache at that index. If they match, it's a hit; if they don't match, it's a miss, & the word is fetched from main memory & replaces the old cache entry.
- Cache can use a block size of one word or multiple words to load entire block on a miss, improving performance due to program locality.

→ Set - Associative Mapping

- Solves the drawback of direct mapping where two words with the same index can't reside inside cache simultaneously
- In set associative cache, each index address can store two / more words under separate tags, forming a set.

TOPIC _____ DATE _____

- When a set is full and a miss occurs, the algorithms like random replacement, FIFO or Least Recently used (LRU) are used to determine which item to replace

→ Writing into cache & initialization

1. Write through method
   - Updates main memory & cache in parallel during every write operation, ensuring that main memory is always valid (crucial for DMA transfers)

2. Write back method
   - only updates the cache location during a write, marks it with a flag & only rewrites it to main memory when that cache word is eventually displaced.

3. Valid Bit

When power is applied, the cache contains non-valid data, a valid bit is assigned to each cache word (initialized to 0) to indicate whether the cache entry contains valid data. It changes to 1 once loaded from main memory.

Pipelining

→ Parallel Processing

- It's a set of techniques used to perform simultaneous data processing tasks to increase speed of system

TOPIC _____ DATE _____

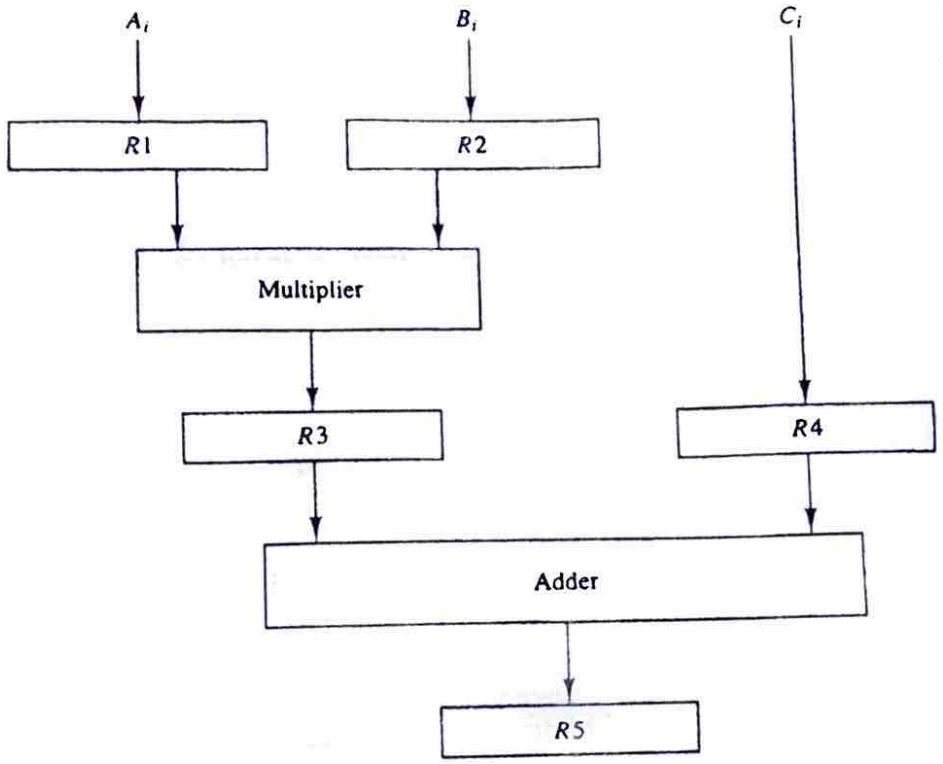
- Instead of executing instructions one by one sequentially like a normal computer, a parallel processing system performs multiple tasks at same time
- While one instruction is being executed at ALU the next instruction can be fetched from the memory / multiple ALU's and processors can run concurrently.
- The primary goal of parallel processing is to speed up computation and increase throughput which is the amount of processing done in a given time interval.
- Higher level parallel processing uses multiple functional units operating in parallel - such as separate circuits for adding, multiplying, shifting and incrementing.

→ Flynn's classification of computer systems

- MJ Flynn classifies computer systems based on the number of simultaneous instruction streams and data streams they handle :-

1. SSID (single instruction, single data stream)
   - Features a single control, memory and processor unit.
   - Instructions are executed sequentially, one after another.
2. SIMD (single instruction, Multiple Data Stream)
   - Features many processing units controlled by a single master control unit
   - All processors get the same instructions but apply it to different data items simultaneously.

Figure 9-2 Example of pipeline processing.



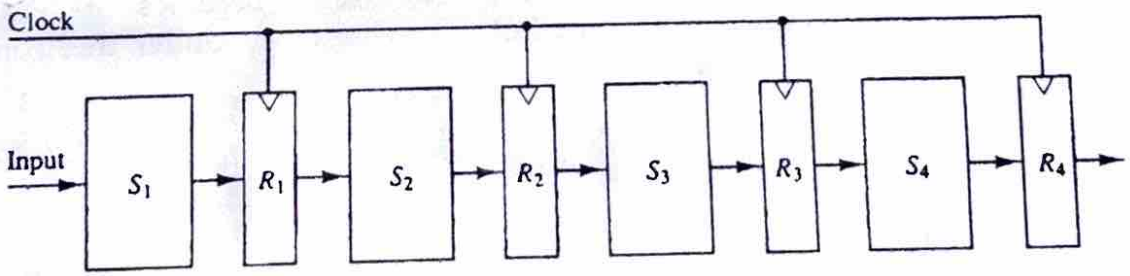


Figure 9-3 Four-segment pipeline.

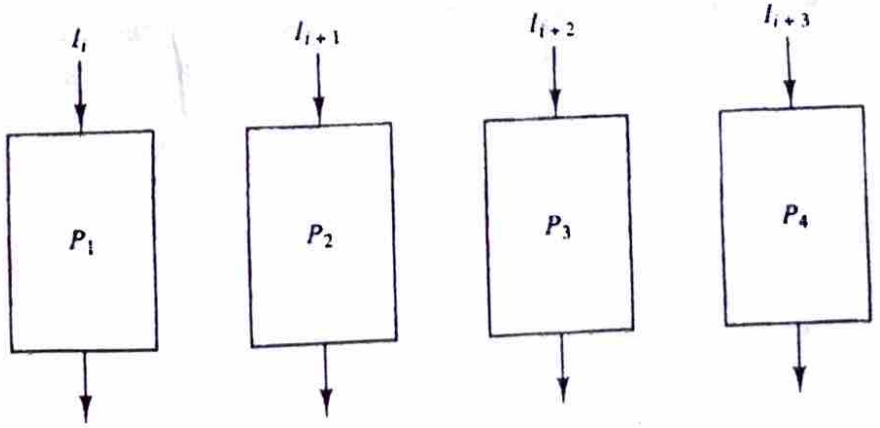


Figure 9-5 Multiple functional units in parallel.

Date:-

3. MISD (Multiple Instruction, Single Data Stream)
   - This structure is purely theoretical, and no practical systems have been built using it.
4. MIMD (Multiple Instruction, Multiple data Stream)
   - Refers to computer systems, like multiprocessors capable of processing several different programs simultaneously.

→ Pipelining
- Pipelining is a technique of breaking down sequential process into subopérations where each subopération is executed in a dedicated segment that operates concurrently with all other segments.
- Multiple computations are in progress at different segments simultaneously.
- Each segment consists of an input register followed by a combinational circuit. Registers isolate segment so each can process distinct data at same time using a common clock pulse.

1. Arithematic Pipeline: Divides arithematic operation into sub-operations for execution across pipeline segments.
2. Instruction Pipeline: Overlaps the fetch, decode and execute phases of the instruction cycle.
   - Different segments may take varying amounts of time to complete their suboperations. The clock cycle must be set to match segment with the maximum propagation time, causing other segments to waste time waiting. Thus, the maximum speed isn't always reached.

→ Example of pipeline processing
Consider $A_i * B_i + C_i$ for $i = 1, 2, \ldots, 7$
- Segment 1 inputs A_i & B_i into R_1 & R_2
- Segment 2 multiplies R_1 & R_2 & store product in R_3 while C_i is stored in R_4

Lakeer

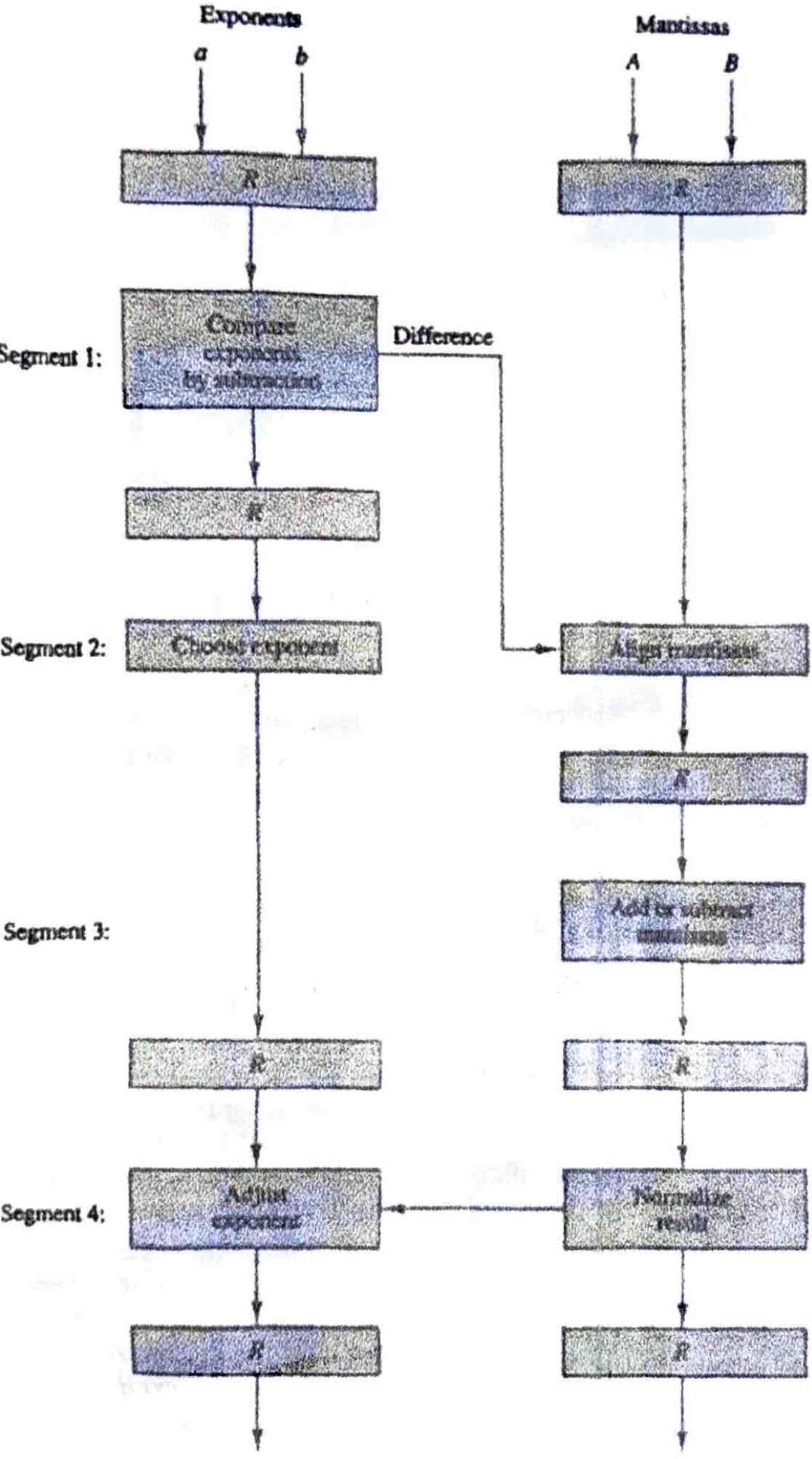


Figure 9-6 Pipeline for floating-point addition and subtraction.

Date:-

- Segment 3 adds R_4 to R_3 and stores final result in R_5.
- It takes 3 clock pulses to fill the pipeline and get the first output, and after that every clock pulse produces new output as long as input data keeps flowing.

→ Arithmatic Pipeline

- Arithmatic pipelines are typically used in very high speed computers for floating point operations and fixed point multiplication
- The pipeline multiplier is structured as an array multiplier equipped with special adders designed to minimize carry propagation time
- Represented as $X = A \times 2^a$ and $Y = B \times 2^b$ where A & B are mantissas (fractions) & a and b are exponent

→ Adder Subtractor floating point pipeline

- Floating point addition and subtraction are broken down into 4 sequential pipeline segments separated by registers R

1. Segment 1 : Compare exponents
   - Compares the two exponents by subtracting them (a-b) to find their difference.
2. Segment 2 : Align mantissas
   - Choose the larger exponent as result exponent
   - Uses the exponent difference to shift smaller mantissa to right, aligning two mantissas under same exponent
3. Segment 3 : Add / Subtract mantissa
   - Adds / subtracts the aligned mantissas together.
4. Segment 4 : Normalize Result
   - Normalizes the final result so that the fraction has a non zero first digit.
   - Adjusts the exponent if an overflow / underflow occurs during shifting

Date:-

→ For eg $X = 0.9504 \times 10^3$ and $Y = 0.8200 \times 10^2$
$A = 0.9504$ $B = 0.8200$ $a = 3$ $b = 2$

1. $3 - 2 = 1$
⇒ larger exponent 3 is selected

2. Mantissa of Y is shifted right changing Y to
0.0820×10^3
⇒ $B = 0.0820$

3. Mantissas are added together producing
$Z = 1.0324 \times 10^3$

4. Sum is normalized by shifting mantissa once to right & incrementing exponent by 1 resulting $Z = 0.010324 \times 10^4$

- This is ~3 times faster than an equivalent non pipeline adder.

→ Instruction Pipelines
- An instruction pipeline reads consecutive instructions from memory while previous instructions are executing in other segments, causing the fetch and execute phases to overlap.
- A basic 2 segment instruction pipeline uses FIFO buffer queue to store instructions sequentially while waiting for processing.
- If a branch instruction occurs out of sequence the pipeline must be completely emptied & all previous read instructions must be discarded

→ General Instruction Cycle
- A complete instruction cycle traditionally involves 6 steps :-

1. Fetch instruction from memory
2. Decode instruction

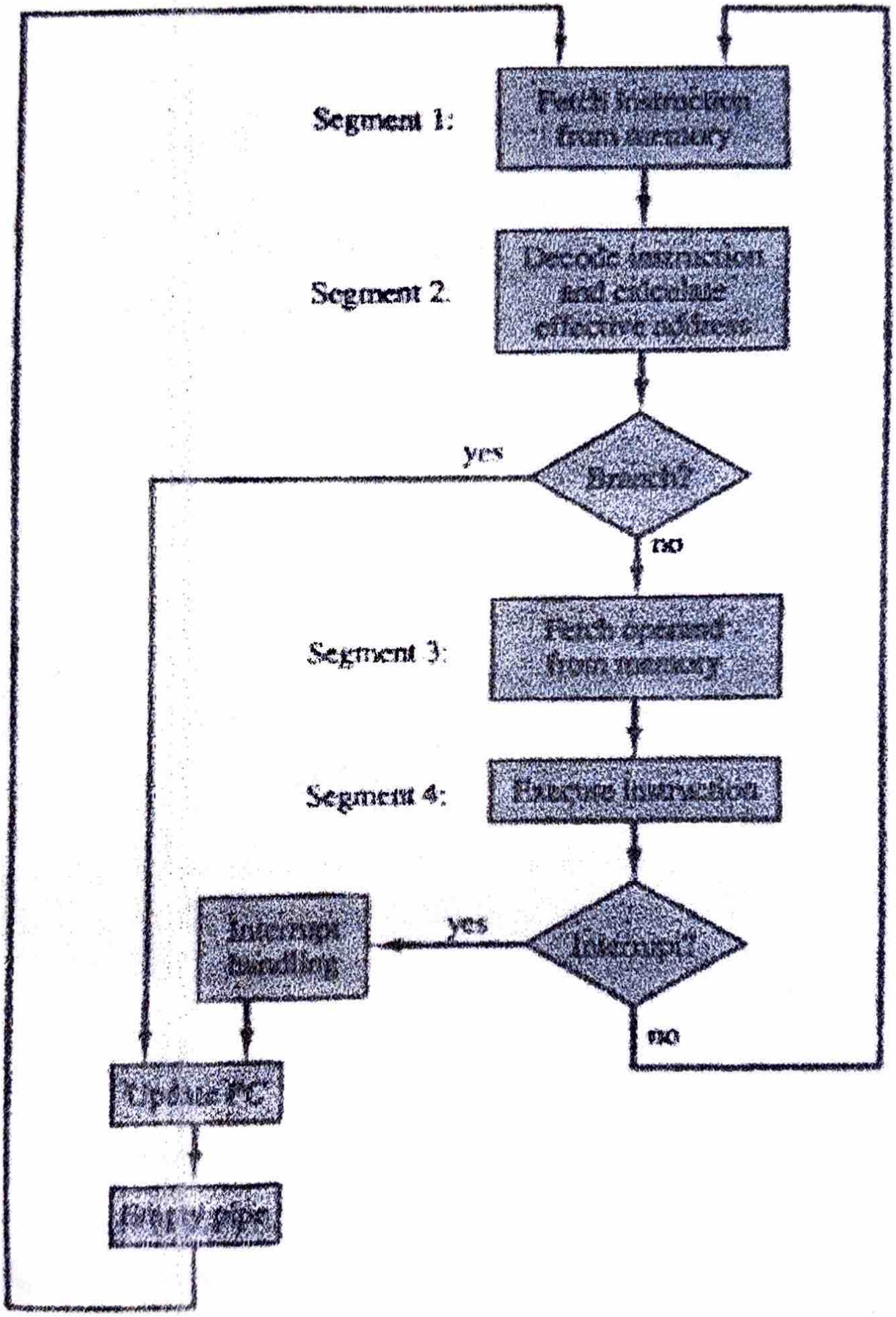


Figure 9-7 Four-segment CPU pipeline.

Date:-

3. calculate effective address
4. Fetch operands
5. execute Instructions
6. store results

- To optimise execution, steps are combined into four active segments

1. FI (Fetch instruction)
   - Fetches the instruction from memory.
2. DA (Decode & effective Address)
   - Decodes instruction & calculate effective address
3. FO (Fetch operand)
   - Fetches operand from memory
4. EX (execute instruction)
   - executes the instruction & stores the result

- Memory access conflicts between fetching instruction and data are solved by using different data memory & instruction modules.

→ Major pipeline conflicts

1. Resource conflict
   - occurs when two segments try to access the memory simultaneously
2. Data Dependency
   - occurs when an instruction requires data or a result from previous instruction that is not ready.
3. Branch Difficulties
   - Arises when branch / interrupt instructions alter the program counter value breaking the linear flow of instructions.

→ Techniques to handle Data Dependencies

- Computers use several methods to resolve data dependency conflicts :-

1. Hardware Interlocks : special circuits detect

Lakeer

Date:-

data conflicts and automatically insert delay clock cycles until the required data becomes available

2. Operand Forwarding: Bypasses the register file by routing ALU results directly through the special hardware paths to next instruction segment
3. Delayed Load: The compiler analyzes code, detects conflicts, and re-orders instructions or inserts no-operation (NOOP) instructions to bridge the delay.

→ Handling Branch Instructions

- To minimize performance delays caused by branching, systems use various strategies :-

1. Prefetch target instructions
Fetches both the target branch instruction & sequential instruction until the final branch decision is made.
2. Binary Target Buffer
An associative memory inside the fetch segment that stores recently used branch addresses and their targets for quick access.
3. Loop Buffer
A high speed register file that stores entire program loops so they can execute directly without repeatedly accessing main memory
4. Branch Prediction
Guesses whether a conditional branch will be taken before it executes to prevent waiting time
5. Delayed Branch
Frequently used in RISC processors where the compiler rearranges code / places useful instructions right after a branch so the pipeline keeps running without interruption.

Lakeer

Date:-

→ RISC Pipeline

- RISC features a simple instruction set where all data manipulation instructions use register to register operations, meaning operands are already in registers & don't need memory fetching.
- It has a 3 segment structure because of its simplicity :-

1. Instruction Fetch : Fetches the instruction from the program memory.
2. ALU operation : Decodes the instruction & performs ALU operations like data manipulation, address calculation for load / store or branch checking
3. Execute Instruction : Directs ALU result to destination registers, data memory / updates program counter

- To prevent conflicts between fetching instructions and accessing data, RISC system uses 2 separate buses with two separate cache memories operating at same speed
- RISC achieves the goal of nearly executing every instruction in a single clock cycle unlike CISC which requires multiple clock cycles.
- In RISC processors, compilers used a technique called delayed branch. The compiler analyses code around the branch and rearranges the instructions - moving useful instructions from before the branch into the delay slot - so the pipeline keeps running continously without interruptions. If no useful instructions are found a NOOP instruction is inserted.

Vector Processing

conventional computers takes days / weeks to solve massive scientific problems which

Date:-

require handling vast number of computations
- Vector processing deals with problems formulated as vectors & matrices using floating point numbers.
- A vector V of length N is an ordered set of one dimensional array of data items represented as $V = [V_1, V_2 \cdots V_n]$
- Instead of using loops like scalar processors vector processors can specify operations on entire array using single composite instruction
eg → C (1:100) = A (1:100) + B (1:100)

→ Memory Interleaving
- Pipeline & vector processors need simultaneous access to memory for fetching instructions and multiple operands at same time
- Instead of single memory bus, memory is partitioned into multiple memory modules each with it's own address & data registers
- In interleaved memory system, consecutive addresses are assigned to different memory modules, this allows the system to stagger memory access cycles and fetch multiple operands from different modules simultaneously

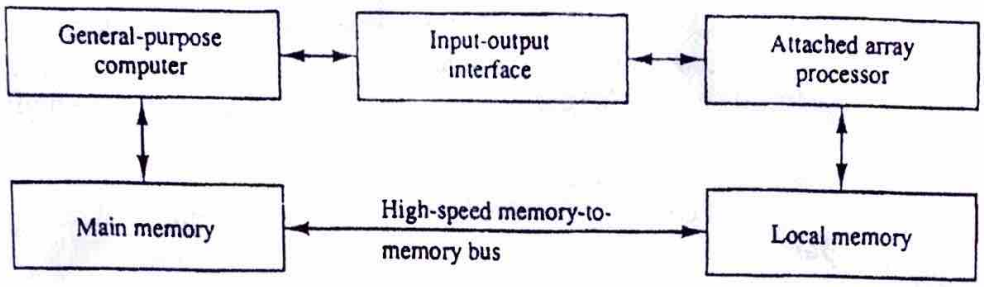
Array Processors

- An array processor performs computations on large data arrays. The term generally refers to two distinct types of processors :-

1. Attached Array Processors
2. SIMD Array processors

- Although both types handle vector operations their internal hardware organisations differ completely.

Figure 9-14 Attached array processor with host computer.



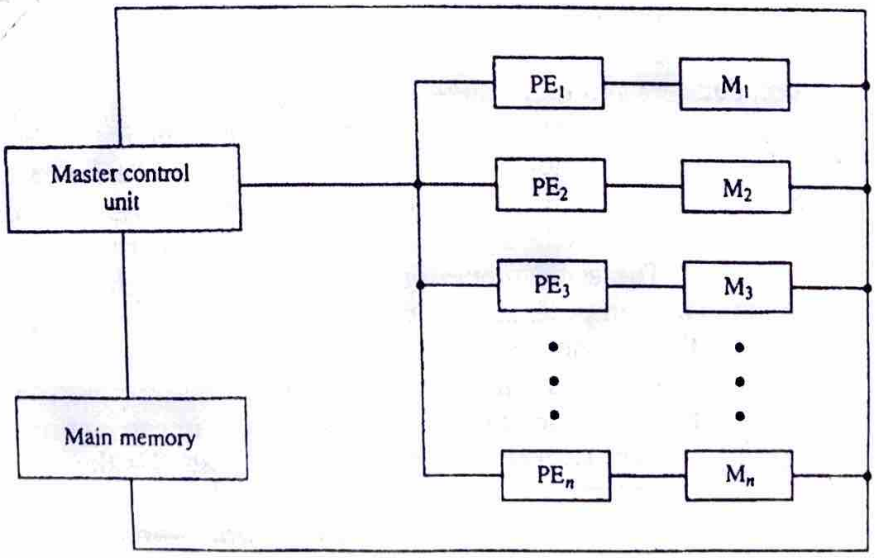


Figure 9-15 SIMD array processor organization.

Date:-

→ Attached Array Processor

- Designed as an auxillary peripheral connected to general purpose host computer to enhance its performance in complex scientific application and vector calculations at a fraction of cost of supercomputers.
- Connected via an input - output interface & a high speed memory bus.
- Features an arithematic unit containing one or more pipelined floating point adders & multipliers

→ SIMD Array Processor

- A computer featuring multiple processing units operating in parallel under the control of single master control unit. (Single Instruction Multiple Data stream array processor)
- Contains set of identical processing elements (PE) where each PE has its own local memory M, an ALU, a floating point arithematic unit & working registers
- Vector instructions are broadcast from master control unit to all PE's simultaneously so that operations take place across all data items in a single cycle
- It uses a flag in each PE to keep active only the necessary PE's when processing vector shorter than the total number of processing elements

Multiprocessors

A multiprocessor system is an interconnection of two or more CPU's (processors) sharing memory and input - output equipments classified as MIMD (Multiple instructions

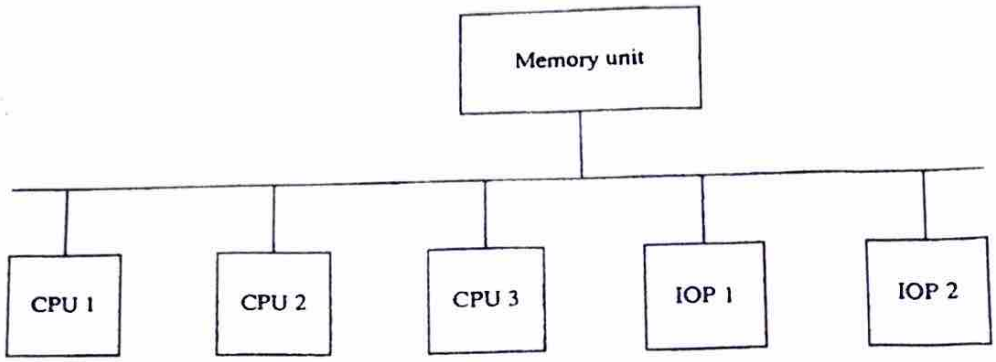


Figure 13-1 Time-shared common bus organization.

Time shared common bus

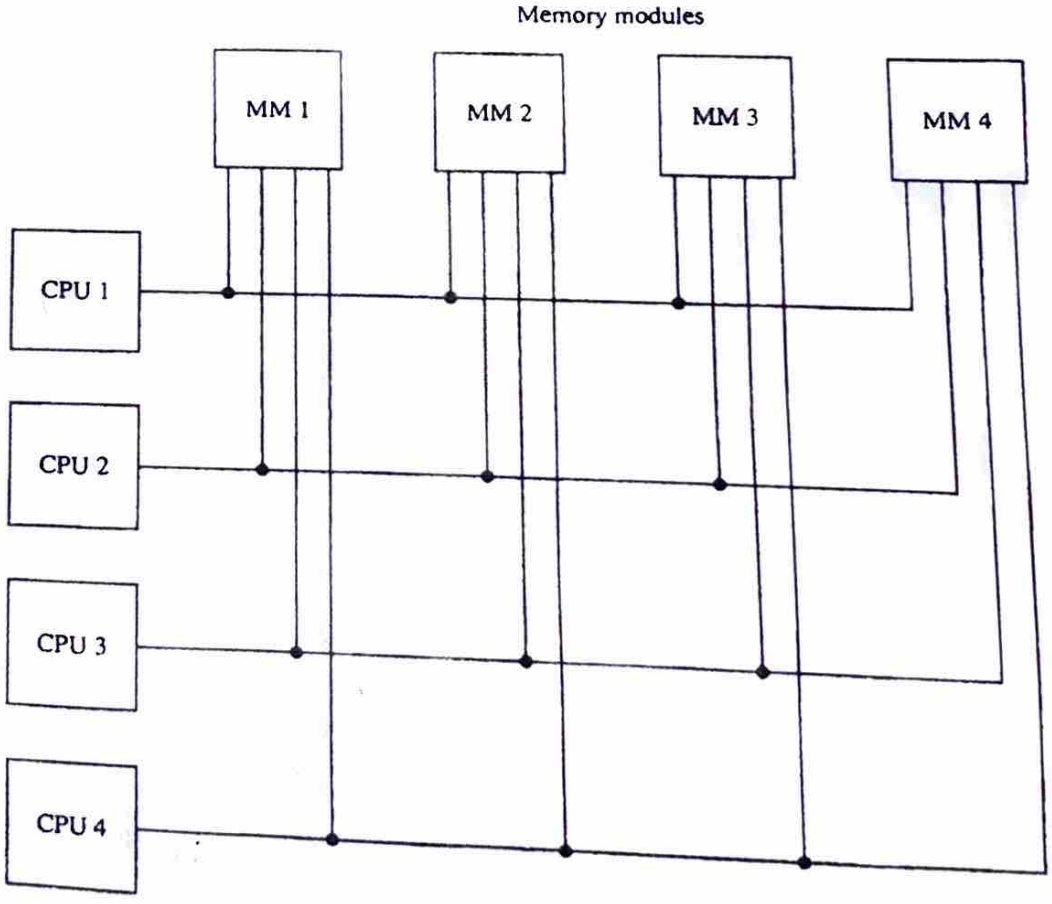


Figure 13-3 Multiport memory organization.

MULTIPORT ORGANIZATION

Date:-

Stream, multiple data stream) systems

→ Advantages of multiprocessors

1. Reliability
If one processor fails, another can take over it's tasks, allowing the system to keep functioning with a minor drop in efficiency

2. High Performance
It's achieved by executing multiple independent jobs in parallel / partitioning a single job into multiple parallel tasks

→ Tightly coupled systems
- Processors share a global common memory though each may have it's own local cache / memory.
- Best for tasks requiring frequent communication

→ Loosely coupled systems
- Each processor has it's own private local memory and communicates by passing messages over a network.
- Best for tasks with minimal interaction

Interconnection Structures

- To connect processors & memory modules multiprocessors use different physical configurations :-

1. Time shared common Bus
- A single common path connects all the

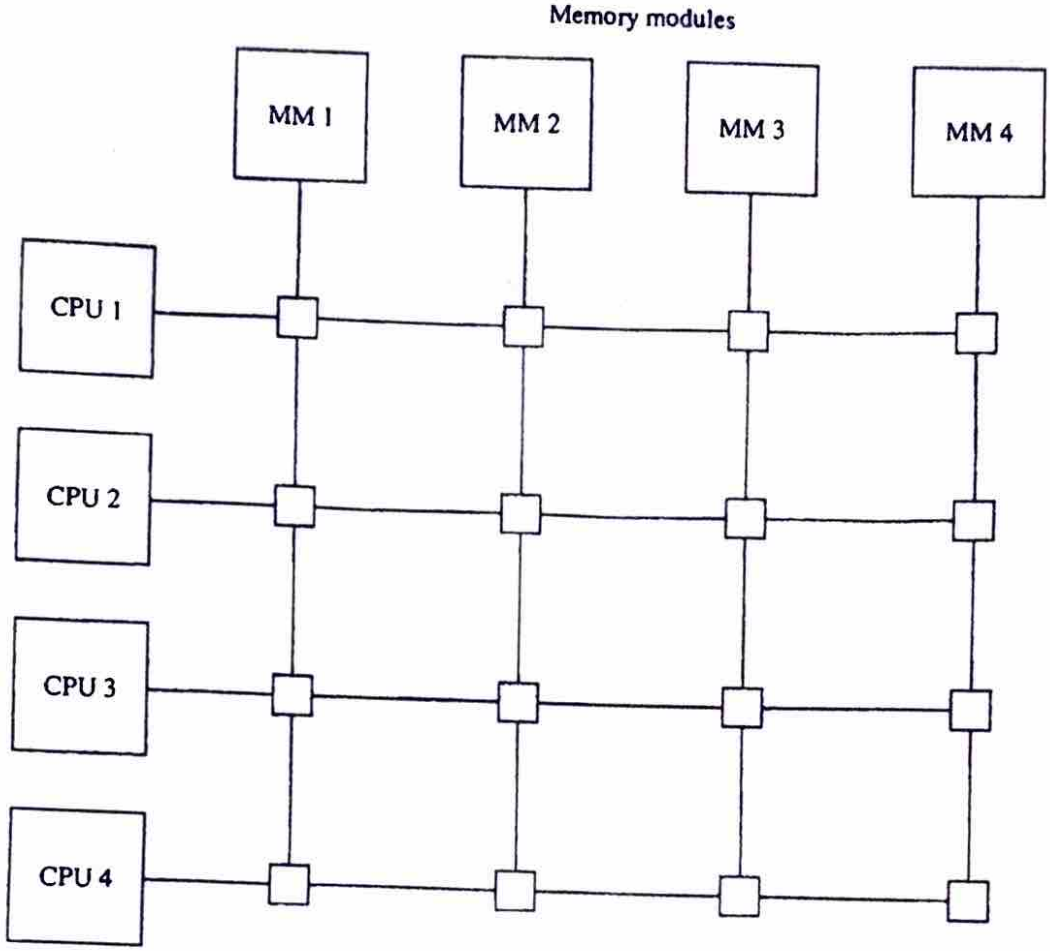
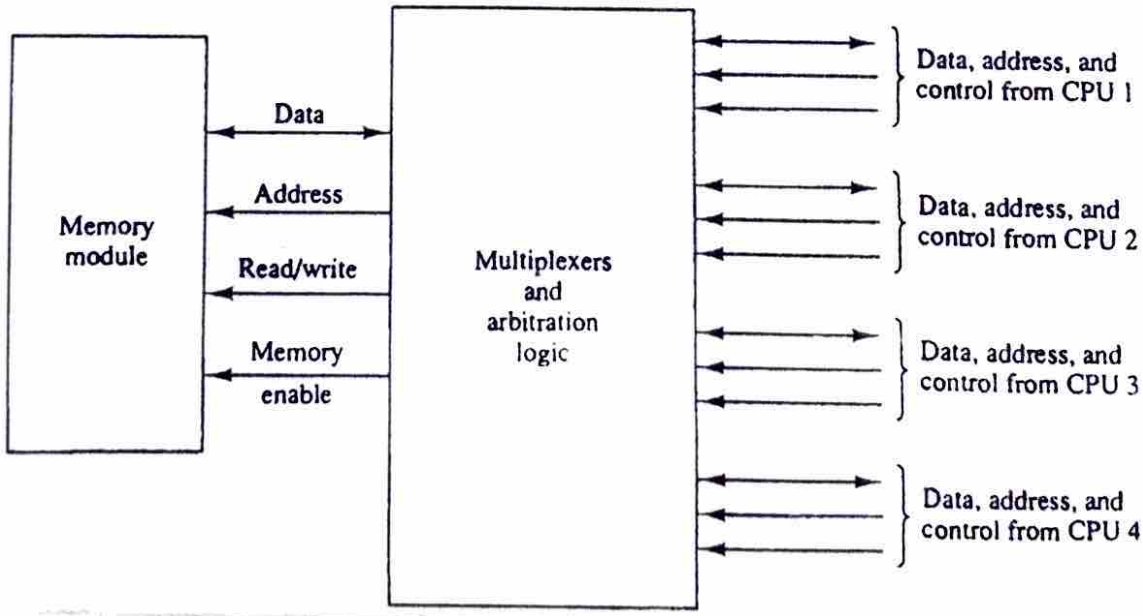


Figure 13-4 Crossbar switch.

Figure 13-5 Block diagram of crossbar switch.



CROSS-BAR SWITCH

Date:-

processors and memory. only one transfer can happen at a time making it a bottleneck for performance

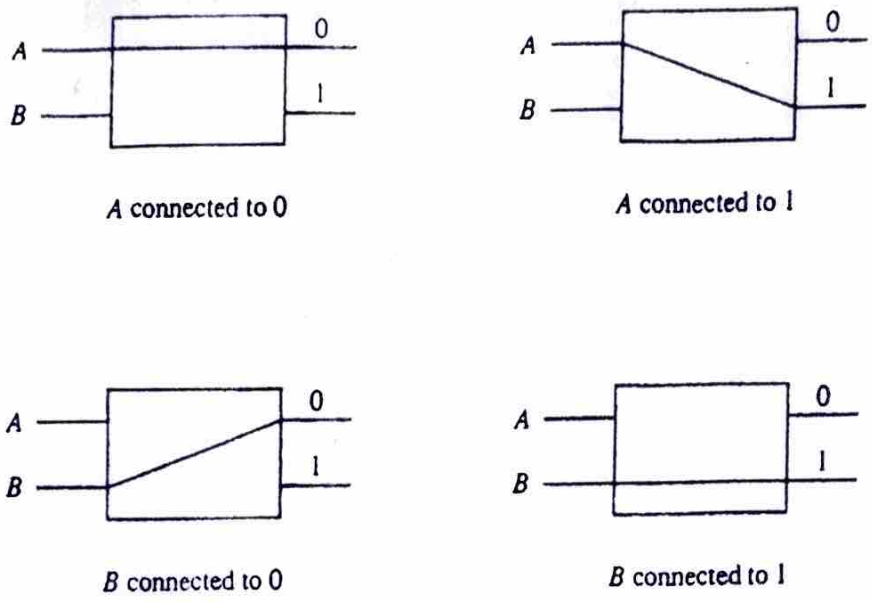
2. Multiport memory
- employs separate buses between each memory module and each CPU. Memory modules have multiple ports with fixed hardware priorities, allowing high transfer rates at cost of complex hardware.

3. Crossbar Switch
- Places electronic switches (crosspoints) at the intersections of processor buses and memory module paths.
- Allows simultaneous transfers across the multiple memory modules, but hardware can become very large & complex

4. Multisage switching Network
- Uses building blocks of 2X2 interchange switches arranged in stages like a binary tree / omega network to route requests from sources to destinations
- each stage examines specific bits of the destination address to route the packet.

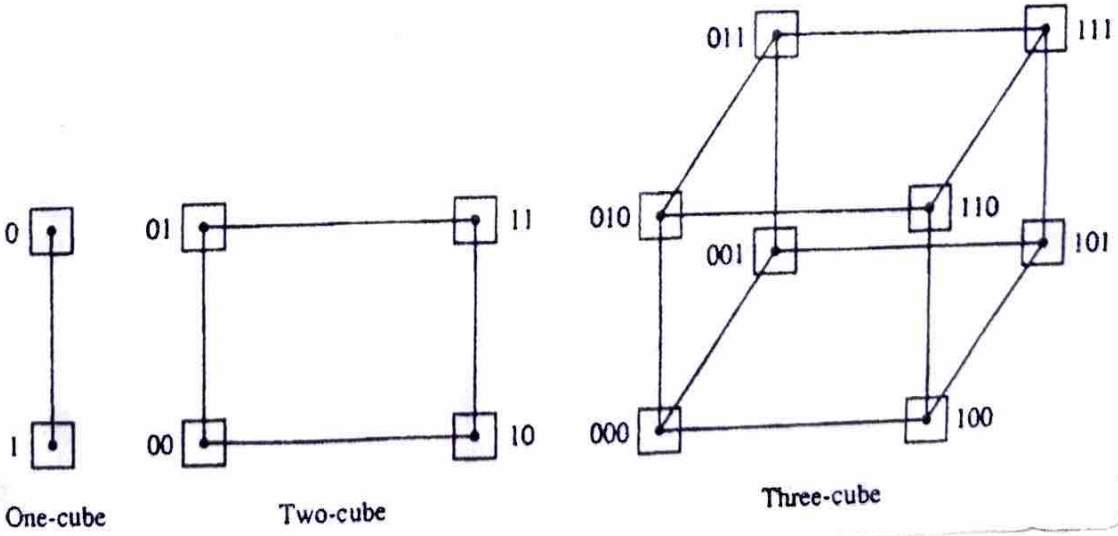
5. Hypercube Network
- A hyper cube / binary N cube is a loosely coupled multiprocessor structure composed of $N = 2^n$ processors connected in an n-dimensional binary cube, where each processor forms a node
- each processor has direct communication paths to n neighbouring processors

Figure 13-6 Operation of a 2 × 2 interchange switch.



MULTISAGE SWITCHING NETWORK

Figure 13-9 Hypercube structures for $n = 1,2,3$.



HYPERCUBE STRUCTURES

Date:-

- Each node has an n bit binary address where the addresses of any two neighbouring nodes differ in exactly one bit position.
- Routing messages between nodes takes from 1 to n links which can be computed using XOR gate of the source & destination addresses

Interprocessor Arbitration

When multiple processors need access to shared system bus simultaneously, arbitration logic is required to resolve contention & grant access safely.

→ Bus arbitration procedures
Arbitration techniques are split into static & dynamic priority algorithms.

1. Static Priority Procedures
   - Serial Arbitration
     Uses a serial connection of bus arbiters. Priority is determined by position along the control line; the device closest to priority line gets highest priority
   - Parallel Arbitration
     Uses an external priority encoder & decoder to recieve requests lines from arbiters & generate the proper acknowledge signals for the highest priority unit.

2. Dynamic Priority algorithms
   - Allocates fixed length time slices of bus time to each processor sequentially in round robin fashion
   - The bus controller uses poll lines to sequence through and check addresses of each

Date:-

connected device
- Gives highest priority to requesting device that hasn't used the bus for longest interval
- Uses FIFO queue to store requests
- It's a dynamic extension of serial (daisy) chaining where priority shifts dynamically without a serial bus controller.

→ Interprocessor communication & synchronization
- Processors in a multiprocessor system communicate via shared memory structures or through common input-output channels
- In shared memory systems, a portion of common memory acts like a mailbox where 1 processor leaves message from another
- Sending processors can alert receiving processor directly using software initiated interprocessor interrupts
- Operating System organizations

1. Master-slave mode : one master processor runs OS functions while slaves handle normal tasks
2. Separate OS : Each processor executes needed OS routines independently
3. Distributed OS : OS functions float dynamically among available processors

→ Inter Processor synchronization & Mutual Exclusion

1. Mutual exclusion
Ensuring orderly access to shared resources so that only one processor can enter a critical section (a program sequence that must complete without interruption) at a time
2. Semaphore
A binary software controlled flag (0/1) used

Lakeer

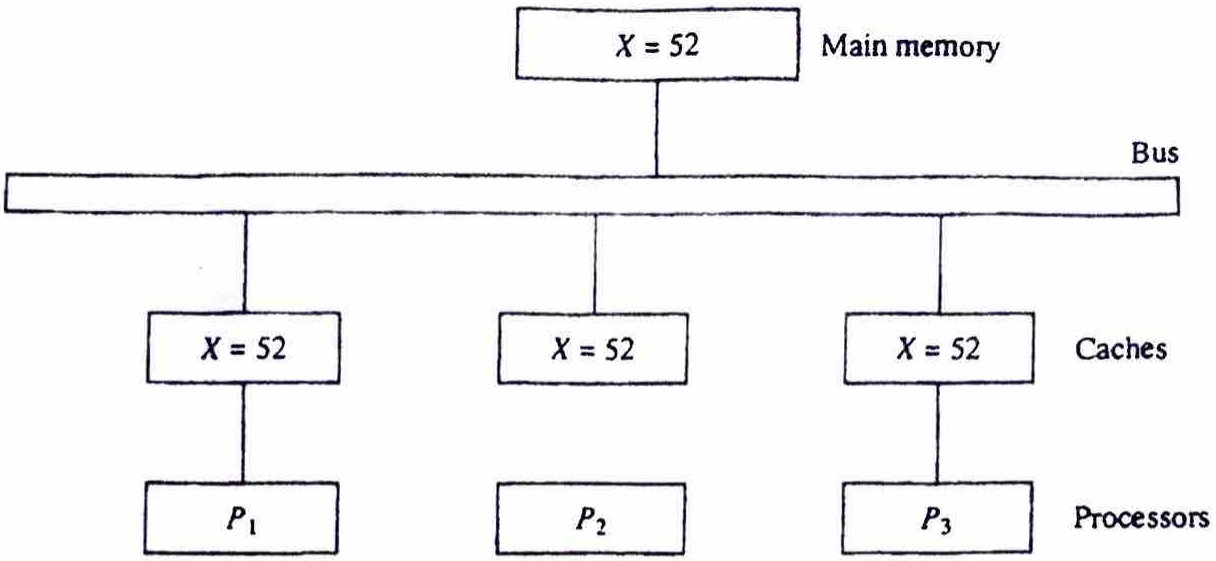
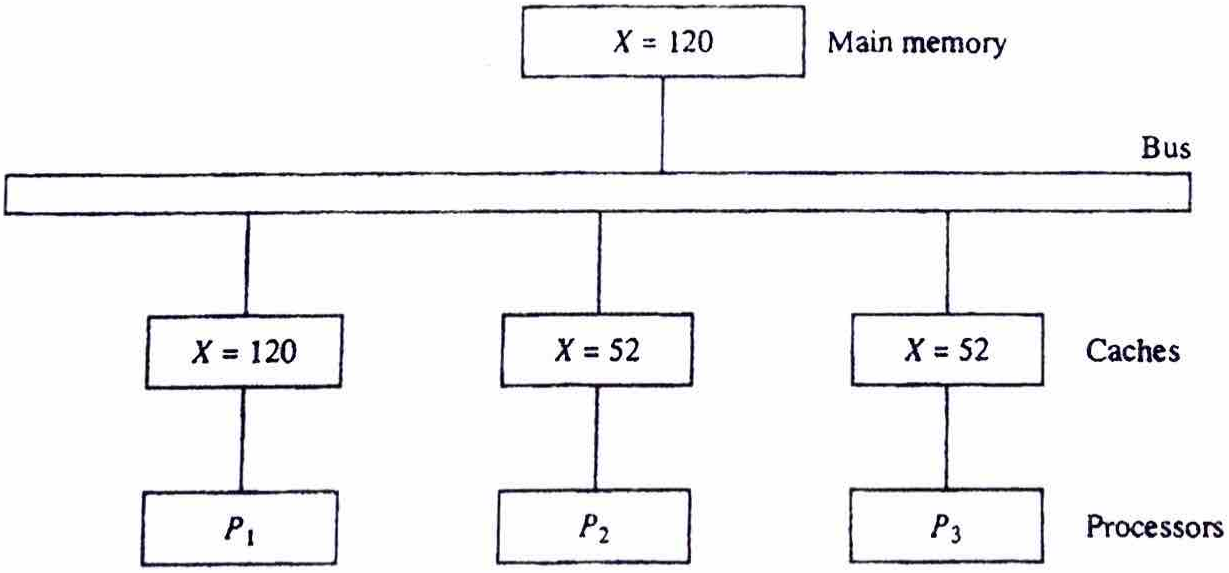
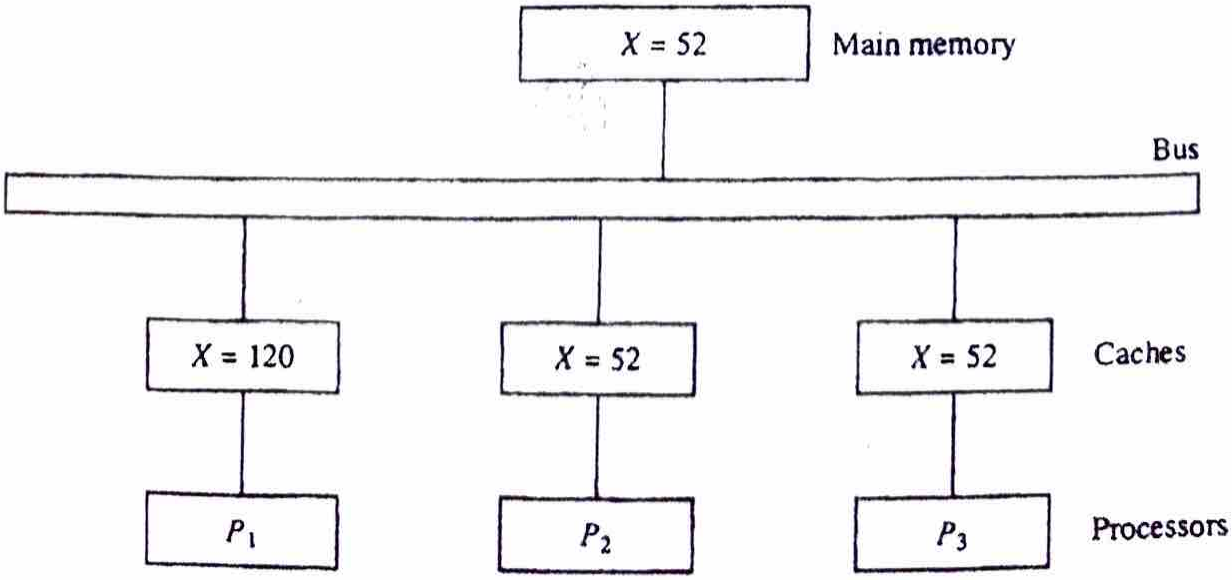


Figure 13-12 Cache configuration after a load on X.

Figure 13-13 Cache configuration after a store to X by processor P_1.



(a) With write-through cache policy



(b) With write-back cache policy

Date:-

to indicate whether a memory segment is currently being used.

3. Hardware Lock & Test And set (TSL)
   - To prevent multiple processors from testing and setting a semaphore simultaneously, hardware locks execute a single indivisible read-write memory operation
   - The instruction tests the semaphore (R ← M[SEM]) and sets it to 1 (M[SEM] ← 1) into two atomic cycles without bus interference

→ Cache Coherence Problem
- In a shared memory multiprocessor with private caches, multiple copies of the same data item (x) can reside in different cache and main memory.
- Incoherence: When one processor updates it's local cache (eg → x changes from 52 to 120) the other private caches & main memory hold outdated values, creating a cache coherence problem
- It occurs due to shared writable data updates or direct memory access by I/O processors.

→ Solutions to Cache coherence
1. Disallowing private caches
   - Using a single shared cache memory associated with main memory, though this violates the physical closeness & increases access time.
2. Catchable v/s Non Catchable Data
   - Compiler tags data so that only non-shared or read only data is cached, while shared writable data remains non-changable in main memory.
3. Snoopy cache controller
   - Its a hardware mechanism where cache

Lakeer

Date:-

controllers constantly monitor ("snoop") system bus write operations.

- If a write occurs to a location held in another cache, the controller automatically updates / invalidates its local copy to maintain complete data consistency.